

Agents, Everywhere

Briefing del equipo. Hackathon global de AI Tinkerers con OpenAI.

Sede San Lorenzo, Paraguay | Build day: sábado 12 de septiembre de 2026 | Documento preparado el 11 de septiembre

Las tres cosas que no podemos ignorar

1. **Tenemos 4 horas 15 minutos de build, no un día.** De 11:15 a 15:30. Todo lo que planeemos tiene que caber ahí.

2. **El video de dos minutos hay que grabarlo durante el build.** Es uno de los cinco entregables obligatorios y es lo único que los jueces globales van a ver de nosotros. Se empieza a las 14:30, no a las 15:25.

3. **El criterio que decide es el 2.** Puntaje 5 significa "un patrón de agente cuyo valor central no podría reproducirse en un cuadro de chat independiente". Si nuestro demo se puede hacer en ChatGPT, ya perdimos.

1. Qué es esto

Hackathon global organizado por la red AI Tinkerers en alianza con OpenAI. Se corre el mismo día en decenas de ciudades más una edición virtual, y **todos los proyectos entran al mismo concurso global**. No hay premio local. Los jueces evalúan después del cierre, sobre lo que subimos al portal.

El tema: "Los agentes están saliendo de la caja de chat. Construí un agente para un lugar donde la gente ya trabaja, habla o vive, y después hazelo significativamente más útil gracias a ese contexto."

Entornos que menciona el brief: web, móvil, Slack, Teams, mensajería, navegadores, software del trabajo, voz, wearables, robótica, o un entorno donde nadie espera encontrar uno todavía.

2. Cronograma del día

Hora	Qué pasa
10:00 - 10:30	Llegada, check in, comida, conocer gente
10:30 - 11:00	Transmisión global de apertura, briefing del desafío y recorrido del starter kit
11:00 - 11:15	Formar equipos y cerrar la idea
11:15 - 15:30	Construir. Esto es todo el tiempo que hay.
15:30 - 16:00	Completar la entrega en el portal
16:00 - 16:45	Show and tell local, opcional, después de haber entregado
16:45 - 17:00	Cierre y foto grupal

Deadline duro del portal: sábado 12 a las 17:00, hora de Asunción. El cronograma local nos pide entregar a las 16:00. Trabajamos contra las 16:00, no contra las 17:00.

3. Los cinco entregables obligatorios

Sin los cinco, el proyecto no es elegible. No son opcionales ni negociables.

1. **Título del proyecto**

2. **Descripción escrita**

3. **Repositorio público de GitHub**

4. **Video demo de dos minutos**

5. **Publicación en redes sociales** etiquetando a los partners del evento

Tres de los cinco entregables no son código. Por eso una persona del equipo se dedica a esto desde las 14:00 en adelante. Es el error más común y el más caro: equipos con buen código que entregan un video improvisado de dos minutos grabado con el celular a las 15:50.

4. Cómo nos van a puntuar

Cuatro criterios, cada uno de 1 a 5. Resumen qué significa sacar 5 en cada uno.

Criterio	Qué mide	Cómo se saca 5
1. Requisitos y funcionalidad	¿El agente funciona de verdad dentro de un lugar donde la gente ya trabaja? ¿El flujo anda de punta a punta?	Sólido, confiable y totalmente funcional dentro de su entorno.
2. Innovación y alineación con el tema	¿Es un lugar o una interacción nueva para un agente? ¿El entorno mejora materialmente lo que el agente puede hacer?	Revela un patrón de agente nuevo y sorprendente cuyo valor central no podría reproducirse en un cuadro de chat independiente.
3. Ejecución técnica e integración	Código, arquitectura, confiabilidad, uso de herramientas, manejo de datos, profundidad de la integración.	Ingeniería excepcional: orquestación sólida, manejo pensado de fallas, arquitectura profundamente integrada.
4. Utilidad y experiencia agéntica	¿Valor claro para usuarios reales? ¿El agente se siente nativo de su entorno?	Desbloquea valor sustancial con una experiencia diseñada específicamente para ese entorno, usando el contexto de forma inteligente y manteniéndose claro y controlable.

Dónde se pierde: el criterio 2 da 1 punto si el proyecto es "esencialmente un chatbot genérico o una automatización, donde el entorno elegido es irrelevante", y 2 puntos si "el entorno funciona principalmente como envoltorio". La mayoría de los proyectos van a caer ahí.

5. Reglas de elegibilidad

Todo proyecto debe ser una construcción nueva hecha durante el período oficial del hackathon.

Sí se puede usar: templates, componentes reutilizables, librerías, prompts, código inicial y otros bloques de construcción. O sea que esta noche podemos dejar listo el entorno, las cuentas, las keys, el repo vacío y los datos de prueba.

No se puede: presentar un proyecto preexistente, ni extenderlo y entrarlo como nuevo. **La funcionalidad central tiene que nacer mañana.** Además, el equipo debe estar preparado para explicar qué partes se construyeron durante el evento. Si nos preguntan, tenemos que poder responder con el historial de commits.

6. Premios

Globales

Puesto	Premio
Primero	10.000 USD en créditos de OpenAI para el equipo, una Mac mini para cada integrante, 1.000 USD en créditos de Exa, swag de Exa
Segundo	5.000 USD en créditos de OpenAI, un par de Ray-Ban Meta para cada integrante, 500 USD en créditos de Exa, swag
Tercero	2.500 USD en créditos de OpenAI, un robot LOOL para cada integrante, 250 USD en créditos de Exa, swag

Categorías especiales

Categoría	Premio
Best Use of Ambiguous AI	Una NVIDIA DGX Spark para el equipo
Best Use of CopilotKit	Un par de AirPods Max moradas para cada integrante

Estrategia: las categorías especiales son nuestra mejor apuesta. En el global competimos contra todas las ciudades del mundo. En "Best Use of CopilotKit" competimos solamente contra los equipos que eligieron usar CopilotKit, que son muchos menos. Si podemos usar un sponsor sin desviarnos del proyecto, lo usamos.

7. Sponsors y para qué sirve cada uno

Sponsor	Qué es	Para qué nos sirve
OpenAI marquee	Modelos y Realtime API	El cerebro del agente. Realtime si vamos por voz.
CopilotKit	Framework para copilotos embebidos dentro de una app, con AG-UI y soporte MCP	Si hacemos un panel lateral dentro de un sitio, esto nos da la UI y nos mete en la categoría especial.
OpenRouter	Gateway a muchos modelos con una sola API	Cambiar de modelo sin tocar código. Fallback si un proveedor falla en vivo.
Exa	API de búsqueda pensada para agentes	Si el agente necesita buscar en la web. Da créditos como premio.
Trigger.dev	Jobs en background, cron y webhooks	La infraestructura exacta del patrón ambiental: agentes que se disparan por eventos en vez de por prompts.
Auth0	Autenticación	Login rápido si el demo lo necesita. No perder tiempo acá.
Ambiguous AI	Plataforma no-code de "AI coworkers" con identidad propia que trabajan en Slack, mail, tareas y documentos	Es el premio más grande de los especiales y probablemente el menos intentado, porque es el sponsor menos conocido.
Mozilla.ai	Herramientas de IA abiertas	Opcional.
Google Cloud Run	Deploy de contenedores	Donde publicamos el backend si hace falta una URL pública.

Al 11 de septiembre el repositorio inicial compartido, los recursos de sponsors, los créditos y la documentación técnica **todavía no estaban publicados** en el portal. Dicen que aparecen antes del build day, probablemente durante la transmisión de las 10:30. No dependemos de eso: sacamos las cuentas gratuitas por nuestra cuenta esta noche.

8. Qué NO construir

Esto sale de mirar qué se presenta en hackathons de agentes recientes. Todas estas ideas sacan 1 o 2 en el criterio 2, porque el entorno es un envoltorio y no algo esencial.

- Bot de Slack que resume hilos o canales
- Agente de email: triage, respuestas automáticas, resúmenes
- Asistente de código genérico o "copilot" de repositorio
- Chatbot RAG sobre documentos o PDFs
- Generador de rutas de aprendizaje o cursos
- Planificador de viajes
- Agente de research que busca en la web y resume
- Agente DevOps que genera Dockerfiles o pipelines de CI
- Tomador de notas de reuniones que arma action items

9. La decisión de fondo: idea local o idea global

El hackathon es global, así que surge la tentación de hacer algo "universal". Vale la pena discutirlo con los números sobre la mesa.

El argumento a favor de lo local

Los jueces van a ver decenas de agentes de productividad genéricos hechos en ciudades con muchos más recursos que nosotros. Lo único que nos diferencia estructuralmente es el conocimiento de dominio que nadie más tiene. Un agente que entiende el SIFEN, o los datos abiertos de la DNCP, no lo puede construir un equipo de San Francisco aunque quiera. El criterio 2 premia exactamente eso: un patrón que sorprende.

El riesgo real de lo local

Que un juez que nunca escuchó hablar de Paraguay no entienda por qué importa, y nos baje el criterio 4, que mide utilidad para usuarios reales.

La salida: encuadre global, ejecución local

No hay que elegir. La facturación electrónica obligatoria existe en decenas de países y sigue expandiéndose. El problema del rechazo de comprobantes por datos maestros mal cargados es universal. Lo presentamos así:

"La facturación electrónica es obligatoria en decenas de países y crece cada año. Cuando un comprobante se rechaza, alguien pierde media hora buscando cuál de cuarenta campos está mal. Construimos el agente para el sistema de Paraguay como implementación de referencia, porque es donde tenemos el dominio, y el patrón se traslada a cualquier régimen."

Eso da lo mejor de los dos mundos: el juez entiende el problema en diez segundos y al mismo tiempo ve algo que ningún otro equipo presentó.

10. Ideas candidatas

Ordenadas por relación entre puntaje esperado y riesgo, considerando que tenemos 4 horas 15 minutos.

A. Agente de navegador que corrige comprobantes antes del rechazo recomendada

- **Entorno:** extensión de Chrome que vive dentro del portal o del ERP donde ya se cargan las facturas.
- **Por qué el entorno es esencial:** el agente lee el formulario mientras la persona lo completa y corrige el campo antes de enviar. En un chat habría que copiar y pegar todo. Este es el punto que nos da criterio 2 alto.
- **Demo de dos minutos:** se carga una factura con la razón social mal escrita, el panel lateral marca en rojo que va a ser rechazada, propone el dato correcto, la persona acepta y el campo se corrige solo. Se envía y pasa.
- **Stack:** extensión con content script que lee y escribe el DOM, panel lateral con CopilotKit, backend liviano de validación, datos de prueba cacheados.
- **Sponsor que suma:** CopilotKit, categoría especial.
- **Riesgo:** leer y escribir el DOM de un portal ajeno de forma confiable. Si a las dos horas no anda, se cambia.

B. Agente ambiental que avisa por mensajería cuando aparece algo relevante plan B

- **Entorno:** Telegram, que se configura en minutos, es gratis y no requiere aprobación.
- **Por qué el entorno es esencial:** el agente no espera prompts. Se dispara por un evento externo, por ejemplo una licitación nueva publicada en los datos abiertos de la DNCP, y aparece solo cuando hay una decisión que tomar. Ese patrón proactivo es lo que puntúa alto.
- **Demo:** aparece una convocatoria nueva, el bot avisa sin que nadie le pregunte nada, con el análisis de por qué nos sirve y cuándo cierra.
- **Sponsor que suma:** Trigger.dev para los jobs y webhooks.
- **Riesgo:** bajo. Es la opción más segura de las tres.

C. Agente de voz en guaraní o jopará alto riesgo

- **Por qué es tentador:** literalmente nadie más en el mundo va a presentar esto. Criterio 2 garantizado.
- **Por qué no la recomiendo con 4 horas:** el reconocimiento de voz en guaraní tiene tasas de error altas y los modelos comerciales no lo cubren bien. Si falla en el video, nos hunde los criterios 1 y 3, que son dos de los cuatro.

Ojo con WhatsApp. Aunque sea el canal dominante acá, la API oficial de Meta requiere verificación de negocio que tarda entre 2 y 10 días hábiles. No sirve para mañana. Si queremos el aspecto de mensajería, usamos Telegram.

11. Roles para cuatro personas

Con 4:15 de build y una persona dedicada a la entrega desde las 14:00, en la práctica tenemos tres desarrollando la mayor parte del tiempo. El alcance tiene que reflejar eso.

Rol	Responsabilidad
P1 Entorno	La integración con el lugar donde vive el agente. Leer el contexto y escribir de vuelta. Es el corazón del criterio 2, va la persona más fuerte.
P2 Cerebro	Backend, lógica de dominio, orquestación del modelo, manejo de errores. Va quien mejor conozca el dominio del problema.
P3 Interfaz	La UI del agente. Si vamos con CopilotKit, esta persona es la que nos habilita la categoría especial.
P4 Entrega	README y repo prolijo desde temprano, guion del video, grabación desde las 14:30, descripción escrita, post de redes etiquetando a los partners. Hasta las 14:00 ayuda con datos de prueba y el entorno del demo.

12. Cronograma de build con puntos de corte

Los puntos de corte son decisiones tomadas de antemano para no discutir bajo presión. Si a esa hora no se cumplió la condición, se aplica el plan de contingencia sin debate.

Hora	Qué tiene que estar pasando
11:15	Idea cerrada. Repo creado. Todos con las keys funcionando.
12:00	Corte 1. El "hola mundo" del entorno tiene que andar: la extensión carga y lee algo del DOM, o el bot responde un mensaje. Si no anda, cambiamos de entorno ahora.
13:00	Corte 2. El flujo central tiene que estar conectado de punta a punta, aunque sea feo y con datos falsos. Si no está, recortamos features hasta que esté.
14:00	P4 sale del código y arranca con el video y la descripción. Los otros tres pulen.
14:30	Primera toma del video, con lo que haya. Nunca quedarse sin video.
15:00	Congelamiento de código. No se agregan features nuevas. Solo se arreglan cosas rotas.
15:15	Toma final del video. Repo público verificado, que abra de verdad desde una ventana de incógnito.
15:30	Subir los cinco entregables al portal.
16:00	Entregado. Recién ahí se respira.

13. Checklist para esta noche

Todo esto está permitido por las reglas, porque no es funcionalidad central.

- Crear las cuentas y sacar las API keys de OpenRouter, CopilotKit, Trigger.dev y Exa. No esperar a los créditos del portal.
- Crear el repositorio público en GitHub, vacío, con README y licencia.
- Confirmar quién hace cada rol.
- Dejar instalado lo básico en cada máquina: Node, el gestor de paquetes, el navegador de desarrollo.
- Preparar el entorno del demo. Si vamos por la idea A, tener el portal o un formulario equivalente listo para grabar.
- Escribir el borrador del post de redes. Mañana solo se le agrega el link.
- Cargar el celular o la cámara con la que se graba el video. Probar el micrófono.
- Entrar al portal y revisar la pestaña de Teams, para ver qué están haciendo los otros equipos de la sede.

14. Enlaces

- Portal del hackathon: `san-lorenzo.aitinkerers.org/hackathons/h_MwGGQrPswqM`
- Handbook: la misma URL con `/handbook` al final
- Equipos: la misma URL con `/teams`
- Sponsors: la misma URL con `/sponsors`
- Mensajes con los organizadores: pestaña Messages del portal

Qué está confirmado y qué es opinión. Las secciones 1 a 7 salen textuales del portal del hackathon al 11 de septiembre. Las secciones 8 a 13 son análisis y recomendación, no reglas. Discútanlas y cámbienlas si no están de acuerdo. Lo que no es discutible es el reloj.

Briefing preparado el 11 de septiembre de 2026 con la información pública del portal. Verificar el portal el mismo día: el starter repo y los recursos de sponsors todavía no estaban publicados.